WEEK 06

Classes, instances and methods

This week: learn, practise, build

01 · Python

Classes, instances and methods
Understand the idea and trace examples.

02 · Practice

Practise the same Python concept.
Predict, change, diagnose and explain.

03 · Project

Persistent plans and line snapshots
Build, discuss and test with a peer.

One weekly component moves the final project forward.

Python foundations

Classes, instances and methods
Concept → trace a small example → explain its behavior

Classes, instances and returned values

Class and instance

Rectangle defines the type; each call creates an instance.

Attributes and self

Each rectangle stores its own width and height.

Method return

area() returns a value the caller can print or test.

Instances share method definitions while keeping their own attributes.

Objects keep data beside related behavior

Class: a definition

A class describes a kind of object. __init__ sets values when an instance is created.

Instance: one object

Counter(4) and Counter(2) create separate objects with separate count attributes.

Method: behavior on an object

first.doubled() uses first as self. A method can read that instance's attributes.

Understand the behavior before changing the example.

Class → instance → method call

1

Define a class

Describe attributes and related methods.

2

Create instances

Each instance holds its own values; an alias shares one.

3

Call a method

Python supplies the receiving instance as self.

Trace which instance receives the call and which attributes its method reads.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
class Name:	Define a type	Group related data and behavior
self.value	One instance attribute	Read or change this object
obj.method()	Call a method	Use the receiving instance
alias = obj	Share an object	Both names refer to one instance

Python Basics · sections 18, 13, 16 · use only when you need a lookup.

Construct one object and read attributes

```
01 class Label:
02     def __init__(self, text):
03         self.text = text
04
05 label = Label("A")
06 print(label.text)
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Call a method and use its returned value

```
01 class Counter:
02     def __init__(self, value):
03         self.value = value
04
05     def next_value(self):
06         return self.value + 1
07
08 counter = Counter(3)
09 print(counter.next_value())
10 print(counter.value)
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Separate instances have separate attributes

```
01 class Counter:
02     def __init__(self, value):
03         self.value = value
04
05 first = Counter(1)
06 second = Counter(2)
07 first.value = 9
08 print(first.value, second.value)
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Two objects share a class, not their attributes

```
01 class Counter:
02     def __init__(self, count):
03         self.count = count
04     def doubled(self):
05         return 2 * self.count
06 first = Counter(4)
07 second = Counter(2)
08 print(first.doubled(), second.doubled())
```

1. Expected: 8 4.
2. Trace which instance is self on each call.
3. Change first.count; inspect both objects.

Predict → execute → explain the value at each step.

Hands-on practice

Week_06.ipynb · Session 2
Predict → run → change → explain

Your turn: practise the same Python skills

P1–P2 · Easy

Use the approach hints. Write the code yourself in a blank cell.

P3 · Core build

Implement a complete small program. Write expected results and assertions.

P4–P5 · Optional

Apply the skill to a new case, then attempt a harder challenge if time permits.

Write your own code, predict the result, then test and explain it.

Check your work before asking for help

A complete implementation

Write the whole operation. Do not stop at one changed character.

Independent expected results

Calculate a normal case and an edge case before running your program.

A peer can explain it

Swap driver and reviewer. Explain one line and one test.

Explain self, separate instance attributes and method calls.

Project studio

Persistent plans and line snapshots
Short design briefing → most time for your work

The people’s task this week

1

Someone needs help

The next volunteer needs to find the plan tomorrow.

2

This week, you will

Save a plan and retrieve it after closing and reopening the database.

3

A useful result

The next volunteer can find the same saved facts.

Use the supplied starting files. Focus on one useful result and check it.

Connect a plan to its batch-line records

Pickup plan

Stable plan ID and explicit planning status.

Offer batch

Stable batch ID with current declaration and availability.

One plan → many lines

One batch → many lines

Plan lines

Container count and historical portions-per-container snapshot.

One plan contains lines; one batch may appear in many plans.

Reference: Define a class with attributes and a method

```
class OfferLine:
    def __init__(self, batch_id,
                 portions_per_container, containers):
        self.batch_id = batch_id
        self.portions_per_container = (
            portions_per_container)
        self.containers = containers

    def total_portions(self):
        return (self.portions_per_container
                * self.containers)
```

1. __init__ sets the instance attributes.
2. self refers to the current instance.
3. The method returns a calculated value.

A class groups instance data and related methods.

Reference: Two instances hold separate values

```
from rescue_models import OfferLine

first = OfferLine("DEMO", 3, 4)
second = OfferLine("DEMO", 3, 2)
print(first.total_portions(), second.total_portions())
assert first.total_portions() == 12
assert second.total_portions() == 6
```

1. Expected planning totals: twelve and six
2. An alias is not a new independent object
3. Public mutation bypasses constructor checks

Python supplies the receiving instance as self.

SQL parameters and a returned row

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE plans(id TEXT, portions INT)")
db.execute("INSERT INTO plans VALUES (?, ?)",
           ("DEMO-P", 12))
row = db.execute("SELECT portions FROM plans WHERE id=?",
                 ("DEMO-P",)).fetchone()
assert row == (12,)
print(row)
db.close()
```

1. Pass values separately from SQL text
2. The one-element tuple needs its comma
3. This memory-only example is not persistence evidence

A one-column result is a tuple; no match is None.

Paths: current directory and resolved filename

```
01 from pathlib import Path
02
03 print("Working directory:", Path.cwd())
04 path = Path("pickup_plans.sqlite").resolve()
05 print("Would open:", path)
06 print("Exists:", path.exists())
```

1. This inspection creates no database
2. A new SQLite path may create an empty file
3. Never reset useful records just to diagnose a path

A relative filename can refer to a different file from another directory.

File persistence: commit, close and reopen

```
from tempfile import TemporaryDirectory
from pathlib import Path
import sqlite3
with TemporaryDirectory() as folder:
    path = Path(folder) / "plans.sqlite"
    db = sqlite3.connect(path)
    db.execute("CREATE TABLE plans(portions INT)")
    with db: db.execute("INSERT INTO plans VALUES (12)")
    db.close()
    db = sqlite3.connect(path)
    print(db.execute("SELECT * FROM plans").fetchone())
    db.close()
```

1. Expected stored result: (12,)
2. Only the newly created practice folder is removed
3. The notebook also uses a fresh reader process

The second connection reads committed file state.

None: handle a missing query result

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE plans(id TEXT, portions INT)")
row = db.execute("SELECT portions FROM plans WHERE id=?",
                 ("unknown",)).fetchone()
if row is None:
    print("Plan not found")
else:
    print(row[0])
db.close()
```

1. A missing row returns None.
2. Use is None to choose the branch.
3. Only index the row when one exists.

Check for None before reading a selected column.

Your implementation TODO

Supplied starting point

Schema, fixture and SQL examples are supplied. Python objects are a modelling example, not a required rewrite.

You implement / do

Save P11 with Q11: 2 × 3 and Q12: 1 × 5. Commit, close, reconnect and retrieve its header and lines; handle a missing lookup.

Check the evidence

Saved total stays 11 after Q11 unit changes 3 → 4; current recalculation becomes 13. Availability stays Q11=10, Q12=4.

Recover the same planning estimate without changing batch availability.

Show the component working

Step	What to show
1 · Locate	Identify the input, this component and its next user.
2 · Implement	Save P11 and line snapshots; reopen and retrieve.
3 · Check	Saved 11; current recalculation 13; stock stays 10/4.
4 · Explain	Show your code, its actual result and why it behaves that way.

Recover the same planning estimate without changing batch availability.

Further reading in English

1. ## Python classes and instance methods

   https://docs.python.org/3/tutorial/classes.html

2. ## Python sqlite3 documentation

   https://docs.python.org/3/library/sqlite3.html

Read one relevant section, then bring an example and a question.